Executive Summary

What it is
BeeYield is a web app for honey traceability, beekeeping operations, and commerce, with a React/Vite frontend and a FastAPI backend.

Who it's for
Primary users appear to be beekeepers/farmers and BeeYield operations staff who record harvests, verify batches, and manage sales and pollination workflows.

What it does

- Public batch traceability lookup (batch code or QR scan) with a journey timeline.
- Harvest, apiary, hive, and farmer CRUD via backend traceability endpoints.
- Local traceability blockchain (HoneyChain) with chain status and recent blocks.
- E-commerce shop + checkout flow with payment methods (M-Pesa and Stripe/card).
- BeeYield dashboards and admin surfaces (frontend routes and backend admin endpoints).
- Precision pollination module (calculator, optimizer, contracts) via backend endpoints.
- AI utilities (e.g., label blurb/label pack generation) via backend endpoints.

How it works (repo-evidence architecture)

- Browser loads React/Vite app (`src/main.tsx`) and routes pages like `/traceability`, `/shop`, `/checkout`, and `/beeyield`.
- Frontend API calls route to the FastAPI backend base URL (`src/services/api.ts` uses `VITE_API_URL` and `/api/v1`).
- Backend entrypoint is `backend/main.py` which mounts `backend/app/api/api_v1/api.py` at `/api/v1`.
- Traceability lookup calls `GET /api/v1/traceability/code/{code}` (`backend/app/api/api_v1/endpoints/traceability.py`).
- Traceability service reconstructs a journey from Supabase tables (harvests, farmers, apiaries, hives) and uses a Rust engine to build a timeline (`backend/app/services/traceability_service.py`).
- HoneyChain blockchain state is stored locally as JSON and exposed via `GET /api/v1/traceability/chain` (`backend/app/blockchain/honey_chain.py`).
- Shop and payments are served by `/api/v1/shop/*` and Stripe endpoints under `/api/v1/payments/stripe/*`.

How to run (minimal)

- Install Node deps: `pnpm install` (see `vercel.json` installCommand).
- Start dev stack: `pnpm run dev` (runs frontend, backend, rust-db, and db-gateway concurrently; see `package.json`).
- Open frontend: `http://localhost:5173/` and backend docs: `http://localhost:8000/docs` (see `QUICK_START.md`).
- Set required environment variables (example: `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_API_URL`; see `.env.example`).

Repo At A Glance

This repo is a multi-service app. The sections below summarize the visible structure without assuming runtime behavior beyond what code/config indicates.

Path	What it appears to contain (repo evidence)
src/	Frontend React app (routes in `src/main.tsx`)
backend/	FastAPI backend, schemas/services, blockchain code
services/	Supporting services (Rust DB service, DB gateway fallback, ML API)
public/	Static assets served by frontend build
supabase/	Supabase-related assets/config (repo evidence: directory exists)
k8s/ terraform/	Infra manifests (repo evidence: directories exist)
vercel.json	Vercel static deploy config for Vite output (`dist/`)
docker-compose.yml	Local compose: frontend, backend, postgres, redis

Key entrypoints

- Frontend: src\main.tsx
- Backend: backend\main.py
- Backend API router: backend\app\api\api_v1\api.py
- Frontend dev config: vite.config.ts
- Dev stack script: package.json (script `dev` runs multiple services)

Frontend Overview (React + Vite)

Routing

- Routes are defined in `src/main.tsx` using React Router.
- Notable routes include `/traceability`, `/shop`, `/checkout`, `/beeyield`, and `/admin` (repo evidence: route declarations).

API client

- `src/services/api.ts` sets `AI_API_URL` from `VITE_API_URL` (default `http://localhost:8000/api/v1`).
- Requests choose a base URL with `getBaseUrl()` (Python backend vs gateway).
- Auth headers are pulled from Supabase sessions (`getAuthHeaders()`).

Traceability UI

- `src/pages/Traceability.tsx` fetches trace data via `traceBatch()` (frontend service).
- It also runs a client-side verifier (`TraceabilityVerifier`) and supports QR scanning (`html5-qrcode`).
- The page offers PDF download via `@react-pdf/renderer` (repo evidence: imports).

State and providers

- App wraps pages with providers for auth, cart, wishlist, theme, language, settings, and React Query (`src/main.tsx`).

Backend Overview (FastAPI)

Entrypoint

- `backend/main.py` creates the FastAPI app and mounts the v1 router with prefix `/api/v1`.
- CORS is configured in `backend/app/core/config.py`.

API surface (high level)

- Routes are registered in `backend/app/api/api_v1/api.py` (many endpoint modules).
- Confirmed modules include traceability, shop, payments (Stripe), pollination, AI label generation, admin, iot, and more.

Traceability endpoints

- `GET /api/v1/traceability/code/{code}`: public trace lookup (journey).
- `GET /api/v1/traceability/chain`: blockchain status and recent blocks.
- `POST /api/v1/traceability/harvests|apiaries|hives|farmers`: create entities.

Payments endpoints

- Stripe endpoints live under `backend/app/api/api_v1/endpoints/payments.py` and are mounted at `/api/v1/payments/stripe/*`.

Data And Storage

Supabase

- `backend/app/db/supabase_db.py` implements Supabase REST calls via `httpx` (select/insert helpers).
- Backend settings include `SUPABASE_URL`, `SUPABASE_ANON_KEY`, and optional service role keys (`backend/app/core/config.py`).

HoneyChain (local blockchain)

- `backend/app/blockchain/honey_chain.py` stores chain state on disk as JSON and exposes chain stats.
- Traceability endpoints call into `honey_blockchain` for chain status and history.

Docker compose (local infra)

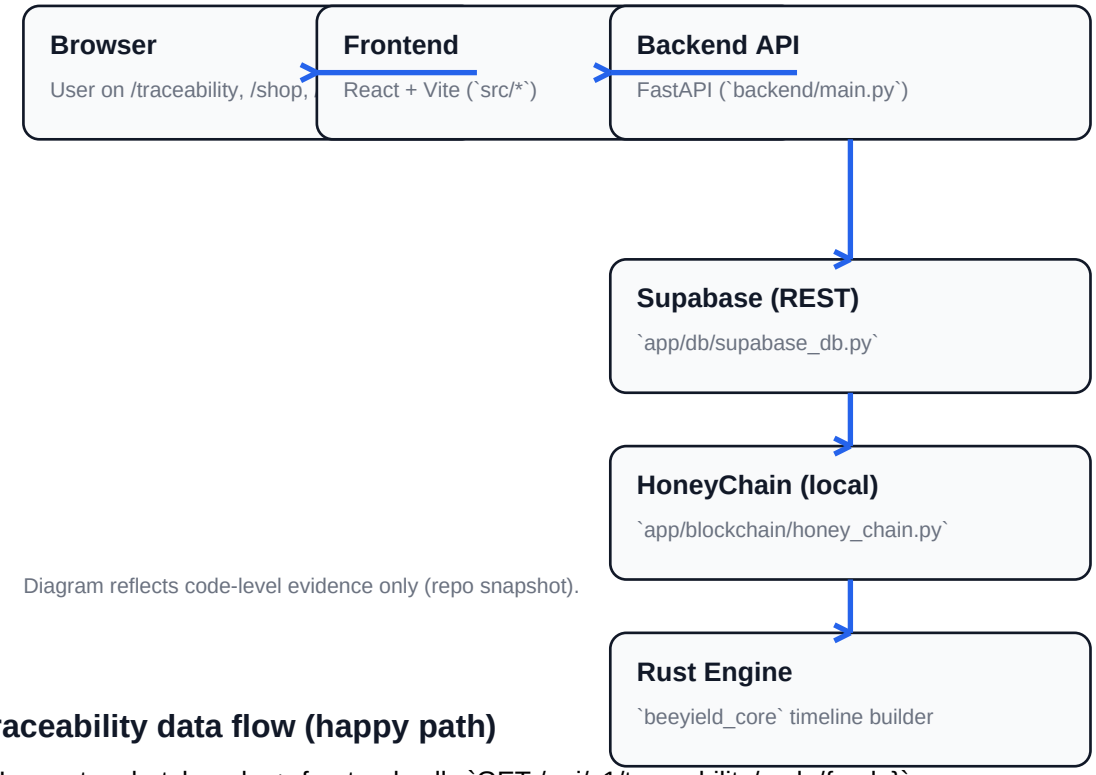
- `docker-compose.yml` defines services for frontend, backend, postgres, and redis.
- Postgres data is persisted via a named volume and migrations are mounted read-only from `backend/migrations`.

Not found in repo

- Backend endpoint `/api/v1/ai/chat` (mentioned in QUICK_START.md) - Not found in backend endpoints files.
- Backend endpoint `/api/v1/ai/status` (mentioned in QUICK_START.md) - Not found in backend endpoints files.

Architecture Diagram (Repo Evidence)

This diagram is derived from code-level evidence only (frontend routes and services, backend routers, traceability service, and blockchain module).



Traceability data flow (happy path)

- User enters batch code -> frontend calls `GET /api/v1/traceability/code/{code}`.
- Backend reads harvest + related entities from Supabase (REST) and returns `TraceResponse`.
- Frontend verifies and renders the journey timeline; offers PDF export.

Shop And Payments

Checkout UI

- `src/pages/Checkout.tsx` implements a multi-step checkout (`cart` -> `payment` -> `receipt`).
- Checkout supports M-Pesa and card (Stripe) based on UI state and service calls.

Shop endpoints

- `POST /api/v1/shop/checkout/init` creates an order and returns payment info.
- `POST /api/v1/shop/checkout/callback/mpesa` receives Daraja callbacks.
- `GET /api/v1/shop/checkout/status/{idempotency_key}` polls for M-Pesa completion status.

Stripe endpoints

- `POST /api/v1/payments/stripe/create-payment-intent` returns `client_secret`.
- `POST /api/v1/payments/stripe/confirm-payment` verifies payment and updates the order.

Configuration (names only)

- Frontend: `VITE_STRIPE_PUBLISHABLE_KEY` (see `.env.example`).
- Backend: `STRIPE_SECRET_KEY`, M-Pesa keys, and related settings (see `backend/app/core/config.py`).

AI And Knowledge Base

AI label generation endpoints

- `POST /api/v1/ai/generate-blurb` returns a short label blurb.
- `POST /api/v1/ai/generate-label-pack` returns a structured label pack.

AI service

- `backend/app/services/ai_service.py` integrates with Google GenAI (`google.genai`) when `GOOGLE_API_KEY` is set.
- The repo also references OpenAI configuration (`OPENAI_API_KEY` in settings and `API_KEYS_SETUP.md`).

Knowledge base

- AI service reads a JSON knowledge base file when present (`backend/app/data/knowledge_base.json`, referenced by code).
- Not found in repo: a clear public API endpoint for `AIService.chat` (QUICK_START.md mentions `/api/v1/ai/chat`).

Deployment And Ops (Repo Evidence)

Vercel

- `vercel.json` configures a Vite static build with output directory `dist/`.
- Client-side routing is handled with a rewrite to `/index.html` for non-`/api/` paths.

Render

- `render.yaml` defines a Python web service (backend) and a static web service (frontend).

Docker

- `docker-compose.yml` provides local dev/prod-like composition including Postgres and Redis.

Infra directories

- Repo includes `k8s/` and `terraform/` directories (contents not summarized here).

Appendix

Minimal dev commands (Windows-first, from repo scripts/docs)

- `pnpm install`
- `pnpm run dev`
- `http://localhost:5173/`
- `http://localhost:8000/docs`

Environment variables (from `.env.example` - names only)

- DATABASE_URL
- MPESA_CONSUMER_KEY
- MPESA_CONSUMER_SECRET
- MPESA_SHORTCODE
- NODE_ENV
- POSTGRES_DB
- POSTGRES_PASSWORD
- POSTGRES_USER
- VITE_API_URL
- VITE_GA4_ID
- VITE_GOOGLE_MAPS_API_KEY
- VITE_GTM_ID
- VITE_STRIPE_PUBLISHABLE_KEY
- VITE_SUPABASE_ANON_KEY
- VITE_SUPABASE_URL

Evidence pointers

- package.json
- QUICK_START.md
- backend\main.py
- backend\app\api\api_v1\api.py
- backend\app\api\api_v1\endpoints\traceability.py
- src\pages\Traceability.tsx
- src\services\api.ts